The Journal of Systems and Software 230 (2025) 112513



Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss





A failure analysis framework to provide pure anomalous data using multi-source data of fault-sensitive microservices☆

Nan Fu, Guang Cheng *, Guangye Dai, Hantao Mei, Xing Qiu, Yue Teng

School of Cyber Science and Engineering, Southeast University, Nanjing, 211189, Jiangsu, China
Jiangsu Province Engineering Research Center of Security for Ubiquitous Network, Nanjing, 211189, Jiangsu, China

ARTICLE INFO

Keywords:
Microservice systems
Multi-source data mining
Feature analysis
Failure detection

ABSTRACT

Proactive failure analysis by selecting critical multi-source data from specific microservices is essential to microservice systems because a system can generate billions of run-time data each day. However, many approaches rely on jointly using multi-source data from all microservices for every fault. In fact, not all faults require such comprehensive data. Besides, many approaches generate false alarms due to neglecting data distribution. Impure anomalous data, containing both normal and anomalous features, can severely hinder root cause localization. Therefore, we propose a novel failure analysis framework to detect failures and provide pure anomalous data generated during failures, which exclusively exhibits abnormal behaviors without overlapping with normal data. Our approach explores the correlation between fault types and multi-source data and analyzes the differences in the sensitivity to sense failures of microservices. It models the representation ability of features from fault-sensitive microservices and the dependencies of these features with various faults. Experiments on two widely used benchmark microservices demonstrate that our approach outperforms other baseline methods, achieving precision of 97.16% and 85.17% and recall of 97.85% and 80.54% on the two benchmark microservices, respectively. The results also provide interpretable information about faulty microservices and key features.

1. Introduction

The migration of enterprise applications to the cloud is a prevailing trend (Zhao et al., 2023). However, the service architecture of large-scale applications is highly complex, involving numerous service components and dynamic interactions. This complexity makes faults in a system inevitable (Liu et al., 2024). An anomaly caused by a fault in one microservice can quickly impact others, significantly degrading the user experience (Lee et al., 2023). Furthermore, since each microservice can run multiple instances, a system can generate billions of run-time records each day (Luo et al., 2021). Therefore, it is crucial to select critical records that facilitate failure detection and localization in large-scale system data, enabling proactive failure detection and localization with a minimal number of records.

Many efforts have been devoted to this end, using a workflow to implement Root Cause Analysis (RCA), as shown in Fig. 1.

First, these RCA approaches extract features that reflect anomalies from multi-source data. They primarily use metrics, traces, or a combination of both for rapid detection. For instance, MicroHECL (Liu et al., 2021) uses response time, error counts, and queries per second metrics for failure detection, MicroRank (Yu et al., 2021) uses latency of traces, while TraceRCA (Li et al., 2021), Grace (Ren et al., 2023), and Nezha (Yu et al., 2023) combine features from metrics and traces. Logs are rarely used in failure detection in the RCA workflow because the complexity of text parsing and high computational costs make them unsuitable for rapid detection.

Next, they use a failure detector to raise alerts. These detectors include traditional solutions, such as SPOT (Siffer et al., 2017) and N-sigma (Lin et al., 2018), as well as machine learning-based (ML) solutions like OmniAnomaly (Su et al., 2019) and SVM. We use public metrics from the Social Network (SN) system by Eadro (Lee et al., 2023) to show that current detectors cannot perform satisfactorily. Specifically, the detection recall for SPOT (Siffer et al., 2017) is 23%, while for OmniAnomaly (Su et al., 2019), it is 35%.

If an anomaly is detected, the data within a predefined time window, deemed anomalous, is fed into the localizer. The purity of these anomalous data is heavily influenced by window selection, closely related to the data distribution. For example, MicroCause (Meng et al., 2020) selects metrics from a 4-h window before the anomaly occurs,

☆ Editor: Jacopo Soldani.

* Corresponding author at: School of Cyber Science and Engineering, Southeast University, Nanjing, 211189, Jiangsu, China.
E-mail address: chengguang@seu.edu.cn (G. Cheng).

https://doi.org/10.1016/j.jss.2025.112513
Received 23 December 2024; Received in revised form 17 April 2025; Accepted 22 May 2025
Available online 7 June 2025
0164-1212/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

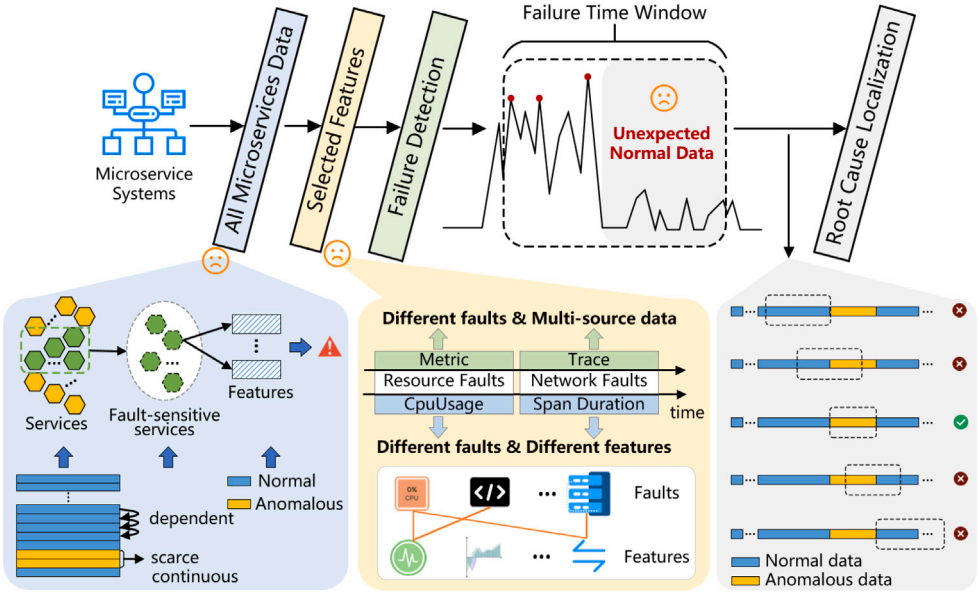


Fig. 1. The workflow for RCA and its challenges.

whereas Dycause (Pan et al., 2023) includes 280 data points after the anomaly occurs.

Then, the localizer identifies the culprit microservices or instances based on the input anomalous data. We compare a traditional root cause localization method, Dycause (Pan et al., 2023), with a deep learning (DL) method, Grace (Ren et al., 2023), to illustrate that as the purity of anomalous data decreases, Grace's F1-score drops from 90% to 60%, while Dycause's top-5 localization performance falls from 71% to 30%. The quality of the anomalous data significantly affects the localizer's performance. If the data lacks true anomalies, the localizer cannot identify the root cause. Conversely, too much normal data can lead to high resource costs and poor localization results.

Therefore, it is crucial for a failure detector to provide high-quality data to the localizer in this workflow. However, current failure detection methods have three limitations that can lead to low purity of the anomalous data.

(1) Weak utilization of the sensitivity differences of microservices to failure detection. Most failure detection methods (Liu et al., 2021, 2020) require analyzing the interactions of all active microservices to detect anomalies. However, when one microservice fails, other microservices exhibit different sensitivities to the resulting anomalies. That is, some microservices are susceptible to the failure of others. These microservices have a high capability to reflect faults, which is determined by the invocation structure among the microservices. Existing approaches often overlook the sensitivity of microservices in sensing anomalies.

(2) Insufficient correlation between different faults and multi-source data. Most failure detection methods (Zhao et al., 2023; Liu et al., 2024; Yu et al., 2023; Panahandeh et al., 2024) for common faults utilize multi-source data. However, they often do not provide a detailed analysis of how features from different data sources relate to various types of faults. Not all faults require multi-source data for detection, and the lack of these analysis can lead to feature redundancy. For instance, our correlation analysis shows that for resource-based faults, such as CPU contention, features from metrics alone are sufficient to reflect the fault. In contrast, for program code faults, such as error returns, metrics alone do not adequately capture the issue.

(3) Inadequate exploitation on the impact of data distribution on failure detection. Most approaches (Meng et al., 2020; Liu et al., 2021; Li et al., 2022), which use a detector and a time window to obtain anomalous data, rely heavily on data distribution. There are significant differences in the selection of window sizes for different datasets. These approaches may generate many false positives, mainly when the anomalous data is scarce and continuous in the test data.

In this work, we propose MSoFSAnomaly, a framework that utilizes **m**ulti-**s**ource data **o**f **f**ault-**s**ensitive microservices to detect **anomalies**, providing pure anomalous data for root cause localization. The novelty of our work lies in combining fault-sensitive microservice selection, feature correlation from multi-source data, and Transformer-based modeling to achieve accurate detection across diverse fault types. The key ideas are (1) selecting fault-sensitive microservices through interaction patterns, (2) analyzing the correlation of different faults and features from multi-source data, and (3) learning discriminative representations of the system status and forcing the model to learn the significant feature changes reflecting anomalies via Transformer-based architecture. Therefore, MSoFSAnomaly can fully exploit meaningful information from multi-source data of fault-sensitive microservices at minimal cost. It is also well-suited for test data with scarce and continuous anomalies, capturing feature changes over adjacent time and providing pure anomaly data for the localizer.

Our main contributions are highlighted as follows:

- We explore the quantitative correlation between different fault types and multi-source data. Based on the analysis of fault propagation patterns, we represent the call relationships between microservices using a hierarchical structure and analyze the differences in their sensitivity to detect anomalies.
- We propose MSoFSAnomaly, a failure analysis framework that detects failures and provides pure anomalous data for root cause localization by leveraging relevant features from multi-source data. It uses Transformer-based architecture to model the feature representations from different fault-sensitive microservices and the dependencies of these features with various faults.
- We evaluate MSoFSAnomaly's performance on two widely-used microservice systems, consisting of 11 and 41 microservices, respectively. The evaluation results show that MSoFSAnomaly provides the purest anomalous data to the localizer, achieving Precision of 97.16% and 85.17%, Recall of 97.85% and 80.54%, and FPR of 3.1% and 8.78% for both datasets. Our implementation of MSoFSAnomaly is publicly available.[1]

[1] https://github.com/FuN312/MSoFSAnomaly.

2. Background

Microservice System and Microservice. The microservice system (Li et al., 2021) is a system that employs the microservice architecture, which breaks the system down into multiple lightweight, loosely coupled, and independent services. These services are called microservices (Weng et al., 2018). For example, Train Ticket (FudanSE-Lab, 2022), a widely used microservice system for train ticket booking, is broken down into 41 microservices, such as order, price, and seat.

Failure and Fault. In a microservice system, a deviation in the service delivery process that is not expected to occur is expressed as a failure, and the cause of this failure is expressed as a fault (Notaro et al., 2021).

Failure Detection and Analysis. In a microservice system, failure detection refers to rapidly identifying anomalous states or behaviors that deviate from expected normal behavior. These anomalies may arise from resource consumption, network delay, or application code errors. Furthermore, failure analysis provides operators with detailed information about the detected failures, such as the anomalous features and specific microservices that reflect anomalies.

Root Cause Analysis and Localization. Root cause analysis is a process of discovering the root causes of failures in order to identify appropriate solutions (Root, 2024). The workflow of root cause analysis contains data collection, feature extraction, failure detection, and root cause localization. After the failure detection, root cause localization analyzes the call relationships between microservices or components and uses the anomalous features to identify the root cause of the failure and its specific location.

Transformer. The Transformer model is a neural network architecture consisting of encoders and decoders. It uses a mechanism called self-attention to capture dependencies between different positions in sequential data. This mechanism allows each input element to attend to all other elements in the sequence, enabling parallel processing of the entire sequence. Unlike recurrent models such as LSTM, which process data sequentially and rely on hidden states, the Transformer can model long-range dependencies and interactions across microservices. One of the key advantages of the Transformer model is its ability to accurately encode long sequential data from microservice systems with test times nearly agnostic to sequence length (Vaswani et al., 2017). For example, when the response time of a microservice becomes anomalous, the self-attention mechanism computes attention scores between this microservice and all others. This enables the model to measure dependencies across microservices. This global perspective helps in failure detection and allows the Transformer to analyze interactions between microservices more effectively than traditional LSTM models. In addition, the attention mechanism provides an explicit representation of dependencies, enabling the model to not only detect anomalies but also identify the critical microservices and features associated with these anomalies. In contrast, LSTM infers all previous timestamps before proceeding further, making them slower and more computationally expensive while also limiting their ability to model long-term dependencies effectively (Audibert et al., 2020). The Transformer model has been successfully applied in existing methods, such as TADL (Li et al., 2023b), demonstrating their effectiveness in capturing complex dependencies for failure detection and root cause localization in microservice systems. Given these advantages, we adopt the Transformer model to capture the evolving features of different microservices over time and finally detect failures.

3. Related work

3.1. Root cause localization-oriented failure detection

In the RCA workflow, failure detection methods play a crucial role by identifying anomalous moments or periods, which serve as inputs for root cause localization, as shown in Table 1. The first type of solutions (Yu et al., 2023; Lin et al., 2018; Ren et al., 2023) focuses on identifying anomalous moments by detectors. In contrast, the solutions for determining anomalous periods typically consist of two stages: anomalous moment detection and window selection. Existing techniques for failure detection in microservice systems can be broadly categorized into statistical-based, model-based, and learning-based methods.

In the anomalous moment detection, statistical-based detection algorithms (e.g., SPOT Siffer et al., 2017, N-sigma Lin et al., 2018) to detect anomalies in microservice systems with the highest probability. N-sigma (Lin et al., 2018) requires the establishment of a separate baseline for the mean and standard deviation for each microservice feature (e.g., CPU usage, response time). SPOT (Siffer et al., 2017) detects sudden changes in time series using extreme value theory. For each time point, it generates a threshold based on the extreme value distributions of past data. These two detectors are highly dependent on data distribution and are sensitive to anomalies. Furthermore, the baseline construction for each microservice feature increases computing and storage costs as the microservice system expands. JumpStarter (Ma et al., 2021) is a metrics-based detector for multivariate time series, such as a combination of CPU utilization, average response time, and requests per second. It constructs a baseline by designing clustering and outlier-resistant sampling algorithms. Compared to SPOT (Siffer et al., 2017) and N-sigma (Lin et al., 2018), it establishes a baseline for each microservice by multiple features. However, while these three statistical-based methods are lightweight for detecting failures, they lack analysis of the correlations between fault types and features. Moreover, they typically rely on strong assumptions about data distributions and cannot effectively model the complex relationships among microservices. As microservice systems grow in scale and complexity, these limitations become more pronounced. To better capture system-wide behaviors, model-based methods that typically construct structured representations have been proposed.

Model-based failure detection methods typically rely on structured information and predefined patterns. They identify anomalies by comparing test data with expected patterns or graph structures. ADSketch (Chen et al., 2022) aims to identify subsequences from metric time series that represent microservice performance issues. To measure how deviated a subsequence is, they calculate its Euclidean distance from other subsequences and search for the smallest distance score. ServiceAnomaly (Panahandeh et al., 2024) jointly uses traces and metrics. It captures the context propagation provided by traces as a graph and annotates this graph by labeling the edges with six metrics. Based on the graph structure, ServiceAnomaly (Panahandeh et al., 2024) compares the test and baseline graphs by sub-graph matching and metric matching to detect anomalies. However, these two methods directly use common features without analyzing the correlation between faults and features. Additionally, ServiceAnomaly (Panahandeh et al., 2024) does not analyze the differences in microservice sensitivity because it relies on comparing the complete call relationships to detect anomalies. While model-based methods incorporate structural information and domain knowledge to improve detection accuracy, they often rely on predefined patterns, handcrafted rules, or similarity metrics. As a result, these approaches may struggle to generalize to unseen fault scenarios or adapt to dynamic system behaviors. To address these limitations, learning-based methods have emerged. These approaches aim to automatically learn complex patterns and feature representations from historical data, offering greater flexibility and adaptability in detecting diverse types of failures.

ML-based (e.g., SVM, Random Forest) and DL-based failure detection methods can analyze correlations between multiple microservice features and use historical data to learn anomaly boundaries. LSTM-AD (Nedelkoski et al., 2019) inputs feature sequences extracted from traces into a long short-term memory (LSTM) model for sequence learning to model the causal relationship between the microservices in a trace. TraceAnomaly (Liu et al., 2020) first construct the service trace

Table 1
Description of failure detectors for root cause localization in the RCA workflow, based on their class (viz., M for anomalous moments detection, P for anomalous periods detection).

Type	RCA methods	Data for failure detection	Detection algorithm	Inputs for root cause localization
P	MicroCause (Meng et al., 2020)	Metrics	SPOT	Metrics within the preceding 4 h when anomalies are detected
P	MicroHECL (Liu et al., 2021)	Metrics	OC-SVM Random Forest 3-sigma	Service calls and their metrics in the current detection window (e.g., 10 min) by using three models to detect anomalies
P	DejaVu (Li et al., 2022)	Metrics	/	Metrics and failure dependency graphs around anomalous moments
M	Grace (Ren et al., 2023)	Metrics, Traces	/	Anomalous metrics (e.g. high CPU utilization) and service traces
M	Nezha (Yu et al., 2023)	Metrics, Traces	N-sigma	Anomalous metrics (e.g., CPU usage rate and Memory Usage Rate) and service call latency at the exact anomalous moment
P	Dycause (Pan et al., 2023)	Metrics	SPOT	Metrics within the current window (e.g., 280) when anomalies are detected
P	CloudRanger (Wang et al., 2018)	Metrics	Polynomial regression functions (Xu et al., 2017)	Metrics in a window (e.g., 5 s) when anomalies are detected
P	MonitorRank (Kim et al., 2013)	Metrics	/	Metrics in an approximate period when anomalies are detected
P	MicroRank (Yu et al., 2021)	Trace	N-sigma	Traces in the last window (e.g., 30 s) when anomalies are detected
M	Microscope (Lin et al., 2018)	Metrics	N-sigma	Metrics and service connections at the exact anomalous moment
P	TraceRCA (Li et al., 2021)	Metrics, Traces	N-sigma	Anomalous traces in the last window when anomalies are detected

vector from a trace, and input these vectors into unsupervised deep Bayesian networks with posterior flow to detect anomalies. However, these methods generally do not fully consider feature changes over time after faults occur. For example, TraceAnomaly (Liu et al., 2020) mainly focuses on the response time series of all spans at a single time point, ignoring feature changes between neighboring time points. This weakness may result in TraceAnomaly (Liu et al., 2020) performing well on a balanced distribution of normal and anomalous data but unsatisfyingly in datasets where anomalous data is scarce and continuous. In addition, these methods extract features from all microservices without selecting microservices based on an analysis of their sensitivity to anomalies.

After the anomalous moment detection, the solutions for determining anomalous periods (Meng et al., 2020; Liu et al., 2021; Wang et al., 2018; Pan et al., 2023) typically select an appropriate window around the anomalous moment according to features of different data. However, Different data has significant differences in the size of this window. This selection may generate many false positives, mainly when the anomalous data is scarce and continuous in the data.

3.2. Features used in failure detection

Table 1 shows that the input for failure detectors contains single-source data (e.g., metrics) and multi-source data (e.g., metrics and traces). Metric-based solutions (Meng et al., 2020; Liu et al., 2021; Wang et al., 2018; Pan et al., 2023) often utilize application-level metrics, including CPU and memory-related metrics, response time, error rates and success rates of requests, queries per second, and throughput-related metrics. Additionally, a trace records the complete lifecycle of a request in microservice systems, while a span represents a specific operation within the trace. Trace-based solutions (Yu et al., 2021) typically use latencies of traces and spans to detect anomalies.

However, some methods (Lee et al., 2023; Yu et al., 2023) have found that metrics and traces provide different insights for different types of faults. While each type of observability data is informative, neither is sufficient to reflect all anomalies. For instance, system-level metrics can effectively indicate CPU contention faults and CPU consumed faults, whereas trace latencies are better at capturing error return faults. Therefore, some solutions based on multi-source data (Yu et al., 2023; Li et al., 2021) enhance fault detection capabilities by combining metrics and traces.

4. Proposed method

4.1. Method overview

We propose MSoFSAnomaly, an unsupervised solution for failure detection comprising training and testing stages. In the training stage, the model learns the characteristics of normal data by minimizing the distance between the normal data and its reconstruction. In the testing stage, the model computes the reconstruction error for the test data. If this error exceeds a threshold derived from the normal data, the data is marked as anomalous. Our approach is based on the idea that the reconstruction error of normal data remains stable within a specific range, while the error for anomalous data may fall outside this range. We focus on establishing a normal boundary using multi-source data from selected microservices. Fig. 2 depicts our design. The workflow of MSoFSAnomaly consists of three phases: fault-sensitive microservices selection, feature exploration with multi-source data, and failure detection and interpretation.

The fault-sensitive microservices selection module identifies the microservices that are easily affected by faults by summarizing the fault propagation patterns under different fault locations to find differences in their sensitivity to detect failures.

The feature exploration module analyzes the high correlation of multi-source data with different faults. MSoFSAnomaly proposes a trace-related feature to reflect the dependencies between multiple spans in a trace. Then, it performs the time alignment for multi-source features and processes the stream of events into continuous feature sequences.

The failure detection and interpretation module uses a sliding window to divide feature sequences into continuous data blocks and encode them. Continuous embedding vectors are input to train a Transformer-based model to capture the difference between each and its reconstructed data block. If the difference exhibits a marked deviation from the normal range, MSoFSAnomaly triggers an alert and interprets the detection results further. This interpretable information provides the faulty microservices and the features with the highest correlation with that fault.

4.2. Fault-sensitive microservices selection

We aim to achieve satisfactory failure analysis using a minimum microservices of systems by exploring fault propagation patterns and selecting fault-sensitive microservices. Since faults propagate along the

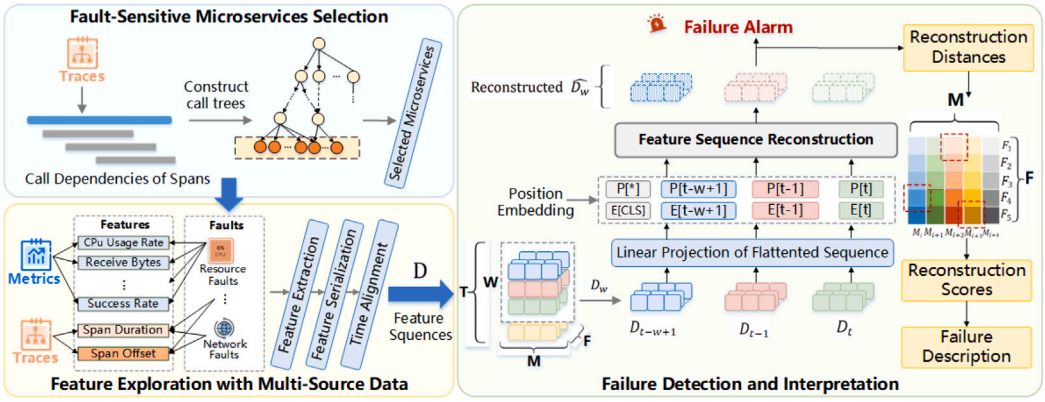


Fig. 2. The framework of MSoFSAnomaly. It is an unsupervised learning approach applied in microservice systems.

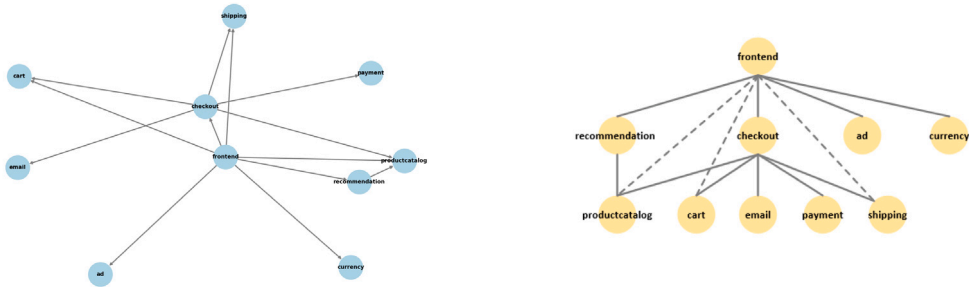


Fig. 3. The conversion from call graph to call tree from normal data of Online Boutique.

call chain of microservices quickly (Lee et al., 2023), some microservices are more affected by other faulty microservices.

To analyze fault propagation patterns, we use traces from normal runs to construct the call relationships between microservices. When there are no circular dependencies, we first construct a graph from these call relationships and then convert it into a call tree structure. This conversion allows us to analyze the depth of microservices in the call chain. Taking the Online Boutique system as an example, Fig. 3 illustrates this conversion process. In Fig. 3(a), the service "frontend" calls both "checkout" and "shipping", and "checkout" also calls "shipping". In this case, we consider "frontend" to be at level i in the tree, "checkout" at level $i+1$, and "shipping" at level $i+2$. We highlight this hierarchical relationship by removing the edge between "frontend" and "shipping".

In the call tree, each node represents a microservice, while the edges represent the calls between them. The nodes can be categorized into three types: the root, a microservice not called by any other microservice; the leaves, which are microservices that do not call any other microservices; and the intermediate nodes, which connect the root and leaf nodes. Fault propagation patterns vary depending on where the fault occurs. As shown in Fig. 4, we categorize fault locations and divide the propagation patterns into four types.

When a leaf node experiences a fault, and if no nodes call this anomalous node, the fault remains confined to that node alone, as shown in Fig. 4(a). If this anomalous node is called by its parents, the fault may affect some of its ancestors and their descendants, as illustrated in Fig. 4(b). When an intermediate node experiences a fault, if no nodes call it, it may propagate along the call chain, impacting some children, as shown in Fig. 4(c). If the anomalous node is called, the fault may follow the pattern in Fig. 4(c) while also affecting some of its ancestors and their descendants, similar to the pattern in Fig. 4(d). We do not treat root nodes as a separate category because their

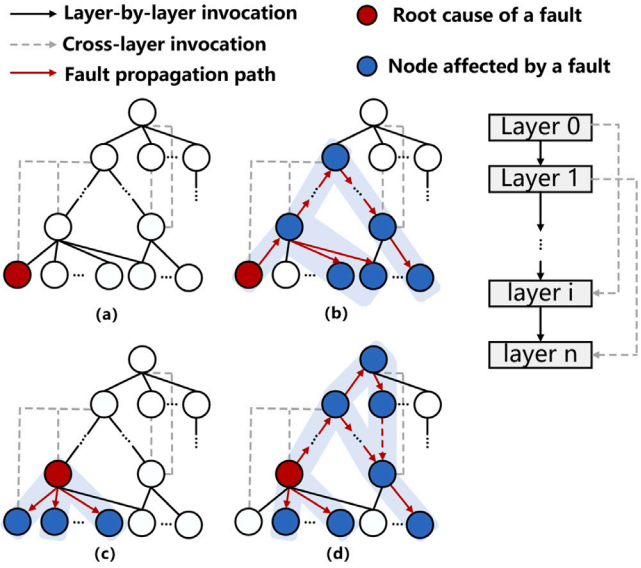


Fig. 4. Fault propagation patterns for different fault locations.

fault propagation pattern is the same as pattern Fig. 4(c). Based on the four fault propagation patterns, faults can quickly spread to leaf nodes along the call chain. We identify microservices that do not call other microservices as fault-sensitive microservices.

In addition, when there are circular dependencies, microservices within a cyclic dependency often exhibit complex failure propagation patterns. Since the microservices in the cycle may depend on each other and create feedback effects, they can not only trigger failures but also

amplify those failures. Therefore, we select all microservices involved in the cycle.

Algorithm 1 outlines the process to select fault-sensitive microservices.

Algorithm 1 Fault-sensitive Microservices Selection

Input: Collected Normal Traces $Tras$
Output: Fault-sensitive microservices M
Initialize M
for $t = 1, \ldots, T$ **do**
  $TraceID, SpanID, ParentID, PodName, OpName \leftarrow aggre(Tras)$
  $call_list \leftarrow invk(TraceID, SpanID, ParentID, PodName, OpName)$
end for
Check for cycles
if no cycles **then**
  $M \leftarrow ExtractLeafMicroservices(call_list)$
else
  $m1 \leftarrow IdentifyServicesInCycle(call_list)$
  $m2 \leftarrow ExtractLeafMicroservices(call_list)$
  $M \leftarrow Merge(m1, m2)$
end if
Return M

The system scale influences the complexity of the selection process. Assuming n microservices and m call pairs are in the call list, we start by checking for circular dependencies using depth-first search, which has a complexity of $O(n + m)$. If a cycle exists, we record the set of involved microservices, which also takes $O(n + m)$. Next, we extract the microservices that do not call any other microservices with a complexity of $O(n+m)$. If no cycle is found, we extract the microservices that do not call others in $O(n + m)$. Therefore, the total complexity for selecting fault-sensitive microservices is $O(n + m)$. In the worst-case scenario, the extracted call list includes all N microservices of a large system, meaning $n = N$. If all N microservices call every other microservice, then m approaches $O(N^2)$. In this extreme case, the maximum complexity of our algorithm is $O(N^2)$. However, in most practical microservice systems, the complexity is typically much lower than this upper limit.

Besides, services can sometimes affect each other indirectly. For example, one service may publish data to a data warehouse, $s3$, or a file while another reads from it. We analyze several widely used distributed tracing tools and find that relying solely on these tools cannot track indirect interactions through shared data storage. This limitation arises because they depend on passing tracing context along the request chain, and in the case of shared data storage, there is no direct request chain to transmit the tracing context.

We discuss how this scenario impacts our microservice selection by an example, which call chain is $C \rightarrow A \rightarrow s3 \rightarrow B \rightarrow D$. Assuming we collect complete trace data, we can extract the call relationship pairs [[C, A], [A, $s3$], [$s3$, B], [B, D]] and choose microservice D as the fault-sensitive microservice. However, our distributed tracing tools currently cannot capture complete trace data. The reason is that the $s3$ step cannot propagate the $TraceID$, resulting in the calls $S3 \rightarrow B \rightarrow D$ being assigned a new $TraceID$. Consequently, we end up with two traces: [[C, A], [A, $s3$]] and [[$s3$, B], [B, D]]. In this case, we select both $s3$ and D. While this selection increases the number of microservices compared to the ideal situation where only D is chosen, it still includes the ideal choice of D. Therefore, it does not negatively impact our failure detection precision.

To enhance the reliability of our selection of fault-sensitive microservices, we utilize a public dataset from a top study (Lee et al., 2023) to demonstrate the failure detection capabilities of various microservices. This dataset includes data from 27 business-related microservices, with normal data collected over 7 h and anomalous data for 1 h at 1-s intervals. Eadro (Lee et al., 2023) randomly injects faults into all microservices every 2 min, with each fault lasting 600 s. We calculate the average features from the normal data as the threshold for each microservice and from the anomalous data to show how different microservices sense failures during the anomaly period. Next, we use the normal traces to extract call relationships, as shown in Algorithm 1, identifying fault-sensitive microservices. Ultimately, we identify 17 fault-sensitive microservices. During the 3600 s of the anomalous stage, we record the number of anomalies reflected by feature changes for all microservices, as shown in Fig. 5.

Fig. 5 shows that among the 27 business-related microservices, the selected fault-sensitive microservice is 17, demonstrating significant failure sense capabilities. Based on the above analysis, our method for selecting microservices is reasonable.

4.3. Feature exploration with multi-source data

The number of metrics or features from a system may overwhelm existing troubleshooting systems (Chen et al., 2020). A fault does not always lead to significant changes in all features. To effectively explore the correlation between faults and features,we first summarize the common faults into three categories: resource faults, program code faults, and network faults (Yu et al., 2023; Pan et al., 2023; Li et al., 2021).

4.3.1. Metrics analysis

Online service systems are often closely monitored using various metrics because metrics provide direct and fine-grained signals that indicate performance issues (Chen et al., 2022). Existing failure detectors typically rely on these metrics to generate alerts. In this section, we analyze 11 system-level and application-level metrics widely applicable to the Kubernetes (K8s) platform.

We use the mutual information (MI) technique (Kraskov et al., 2004) to assess how 11 metrics contribute to detecting three categories of faults. For the metrics collected from two systems: Online Boutique (OB) and Train Ticket(TT), we divide them into chronological blocks around a fault to ensure that each block contains normal and anomalous data. For each block, we calculate how well each metric reflects abnormalities using Eq. (1). Here, X represents the metrics, and Y represents the data category (e.g., normal or anomalous data). The $p(x, y)$ is the joint probability distribution of X and Y, while $p(x)$ and $p(y)$ are the marginal probability distributions of X and Y, respectively. The importance values $I(X; Y)$ range from 0 to 1.

$$I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)} \tag{1}$$

In our experiment, we compute the MI values for each fault individually. This yields, for each fault, the MI values between selected microservices and corresponding features. Then, we average the MI values of the same feature across all selected microservices under that fault. As a result, we obtain an MI score for each feature under each fault. Finally, we select those features with an average value greater than the 90th percentile for all faults. Table 2 shows that the metrics MCUR, NCUR, and NRB provide sufficient information for resource faults (e.g., CPU contention, CPU consumption). At the same time, the MMUR feature offers some insights into network faults (e.g., network delay). However, none of the 11 metrics significantly reflect program code faults (e.g., error returns, exception codes) in either dataset.

4.3.2. Traces analysis

A trace represents the end-to-end journey of a request across various microservices, while a span represents an individual operation within that trace. The duration of each span, referred to as Span Duration in this paper, is used in existing studies to reflect anomalies. However, Span Duration only focuses on individual operations and does not capture the relations between multiple spans. we extract the call relations

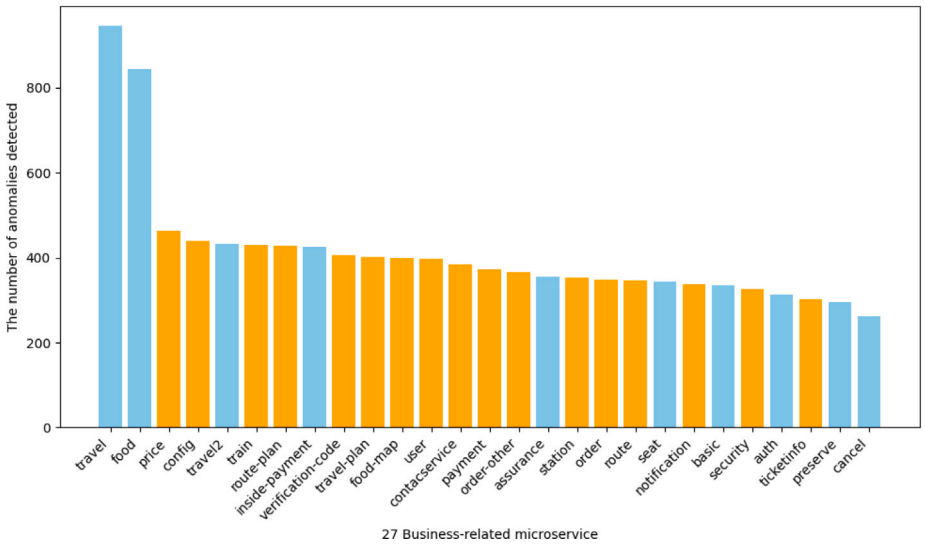


Fig. 5. The number of anomalies reflected by feature changes. It is arranged in descending order from left to right based on the number of anomalies, with orange representing the microservices we selected and blue representing the others.

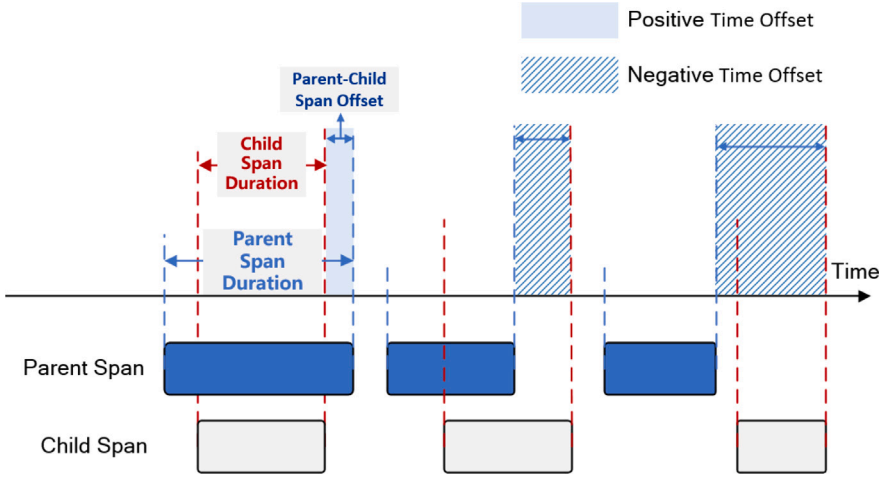


Fig. 6. Span duration and span offset features in a trace.

Table 2
The effective metrics for three faults on two data.

Fault types	OB data	TT data
Resource faults	MCUR[a], NCUR[b], NRB[c]	MCUR[a], NCUR[b]
Network faults	/	MMUR[d]
Program code faults	/	/

[a] MCUR: CPU usage rate of each microservice.
[b] NCUR: CPU usage rate of each node.
[c] NRB: Network bytes received by each node.
[d] MMUR: Memory usage rate of each microservice.

among multiple spans from traces and summarize the three relations between a parent and child span, as shown in Fig. 6. Note that the same relations also exist among multiple child spans.

We propose a feature, called Span Offset, to understand the time distribution and potential bottlenecks between different steps in the operation chain of a trace. This feature captures the dependencies between multiple operations by calculating the relative timing of each child span concerning the end of its parent span. Fig. 6 illustrates that Span Offset can be positive or negative. For example, a positive Span Offset occurs when a parent's span ends after its child's span. We distinguish two types of negative Span Offsets by comparing the absolute value of Span Offsets to the Span Duration of the child span.

To verify the detection capabilities of Span Duration and Span Offset for network and program code faults, we extract them for each microservice and use a version of the N-sigma method to analyze how well these two features can reflect anomalies. Table 3 shows the contributions of Span Duration and Span Offset to detecting three faults, measured by Recall.

We discover that the two features can reveal faults that metrics alone cannot identify. It is important to note that in the above experiment, we sacrifice the detection precision of normal data. Based on the analysis of features from metrics and traces, we conclude that combining features from multi-source data is beneficial for reflecting different anomalies.

Table 3
Performance of span duration and span offset in two microservice systems.

Faults	OB data			TT data		
	Span offset	Span duration	Both features	Span offset	Span duration	Both features
Network delay	100%	42.9%	**100%**	100%	100%	**100%**
Error return	0%	33.3%	**33.3%**	72.7%	100%	**100%**
Exception code	33.3%	33.3%	**66.7%**	100%	84.6%	**100%**

Therefore, for the selected microservices, we use the MCUR, NCUR, and MMUR features to reflect resource faults, while the Span Duration and Offset features are used to reflect program code and network faults. Algorithm 2 provides the extraction procedure.

Algorithm 2 Feature Extraction on Multi-Source Data

Input: Collected Metrics $Mets$, Traces $Tras$ and M
Output: Features of selected microservices F
Initialize F
for $t = 1,\ldots, T$ **do**
  $MCUR, NCUR, MMUR$ = metrics_extra($Mets, M$)
  Sp_Dur, Sp_Off = traces_extra($Tras, M$)
  F = aggre2($MCUR, NCUR, MMUR, Sp_Dur, Sp_Off, M$)
end for

4.3.3. Feature serialization

Metrics collected from fault-sensitive microservices are time series with a serialized structure. Therefore, they only require regular preprocessing steps. However, separately normalizing the training and test sets can lead to significant changes in the distribution of normal data between the two datasets. Therefore, we use Z-score standardization. The standardization process is given by: $M'_d = (M_d - \text{mean}(M_{\text{train}}))/\text{std}(M_{\text{train}})$, where M_d represents the raw data, and M'_d denotes the standardized data, with M_d taking values from both M_{train} and M_{train}. Additionally, mean(M_{train}) and std(M_{train}) refer to the mean and standard deviation of the training set, respectively.

Traces collected from fault-sensitive microservices heavily depends on user behavior, there may be instances where a fault-sensitive microservice neither invokes other microservices nor is invoked by them. In such cases, trace-related features for that microservice may be missing. To address this, we perform missing values imputation. Finally, we apply a standardization process similar to that used for metrics to the two trace-related features.

4.3.4. Clock synchronization and time alignment

To conveniently construct features from the two-source data of all fault-sensitive microservices, MSoFSAnomaly first synchronizes the clocks of the metrics and traces after serialization. We retain features that share the same Unix timestamps, establishing a correspondence between the two modalities at those time points. Additionally, metrics are collected every minute, while traces are recorded when a request is processed, with raw traces aggregated every minute.

In summary, through the feature construction and serialization, we obtain a three-dimensional sequential feature $D_{(T,M,F)}$ composed of the feature F, the microservice instance M, and the time T.

4.4. Transformer-based failure detection and interpretation

MSoFSAnomaly uses a Transformer-based model for failure detection and interpretation. The architecture of this model is illustrated in Fig. 7. It showcases four parts: the embedding of window-wise data blocks, which describes the embedding process for the three-dimensional feature sequences (Section 4.4.1); feature sequence reconstruction, which includes the encoder, decoder, and output layers to reconstruct the raw data (Section 4.4.2); and failure alarm and interpretation, which conducts failure analysis by calculating the reconstruction error based on the distance between the raw data and the reconstructed data (Sections 4.4.3 and 4.4.4).

4.4.1. Embedding of window-wise data blocks

To capture the feature changes from selected microservices, we use a sliding window mechanism to divide the three-dimensional features $D_{(T,M,F)}$ into continuous data blocks. Each data block contains the three-dimensional features corresponding to W time points. Transformer has proven effective in modeling sequential data and has recently been adopted in various fields for failure detection (Hao et al., 2024). In sight of this, we design a modified Transformer model, and utilize the feature sequences from all selected microservices at time t, called D_t, as a token to model the feature changes between adjacent time points within each data block.

The standard Transformer receives as input a 1D sequence of token embedding. For the three-dimensional feature D_t, we flatten it along the feature and microservice dimensions, then use a linear mapping to convert it into two-dimensional time-series features $D'_t \in \mathbb{R}^{W\times(M\cdot F)}$. This step ensures that we retain the failure information in the original data D_t while transforming the features of the selected microservices into fixed-length vectors. Additionally, the positions of the selected microservices and their corresponding features within each vector are fixed, which is beneficial to interpret anomalies. We use the standard learnable 1D position embedding to retain positional information between adjacent time points. The embedding process for D'_t is as follows:

$$z^t_0 = [D'_{t-w+1}; \cdots; D'_{t-1}; D'_t] + E_{\text{pos}}, E_{\text{pos}} \in \mathbb{R}^{W\times(M\cdot F)} \tag{2}$$

where E_{pos} is the position embedding of tokens, and z^t_0 is the raw embedding served as input to the sequence reconstruction.

4.4.2. Feature sequence reconstruction

We utilize a modified Transformer model as a sequence reconstruction module. This module consists encoder, decoder and output layer.

The encoder produces representations based on feature sequence of window size. It consists of alternating multi-head self-attention (MHA) layers and MLP blocks. The MHA block enables the model to focus on information from different positions in the input embedding simultaneously. It does this by splitting the raw input vector into H heads, allowing each head to learn distinct attention weights independently. In this paper, H represents the number of features. The MHA block calculates the attention weights between the Query and Key matrix using the Query, Key, and Value matrix across five heads, employing Scaled Dot-Product Attention.

Subsequently, the MHA block applies the computed weights to the Value matrix, finding important features of the input embedding based on these attention weights. Finally, the MHA block concatenates the weighted outputs from the H heads and generates the final output through a linear projection. Eq. (3) shows the entire computational process.

$$z'_\ell = \text{Drop}(\text{MHA}(\text{Norm}(z_{\ell-1}))) + z_{\ell-1} \quad \ell = 1, 2, \ldots, L \tag{3}$$

The MLP block first applies a linear projection to z'_ℓ. Next, it uses the GELU activation function to introduce non-linearity and applies LayerNorm. Finally, a residual connection is added to obtain the encoder output z_ℓ. The computational procedure is illustrated in Eq. (4).

$$z_\ell = \text{Norm}(\text{MLP}(\text{Norm}(z'_\ell))) + z'_\ell \quad \ell = 1, 2, \ldots, L \tag{4}$$

The objective of the decoder is to reconstruct the feature sequence based on the encoder output. Our approach uses five-head self-attention

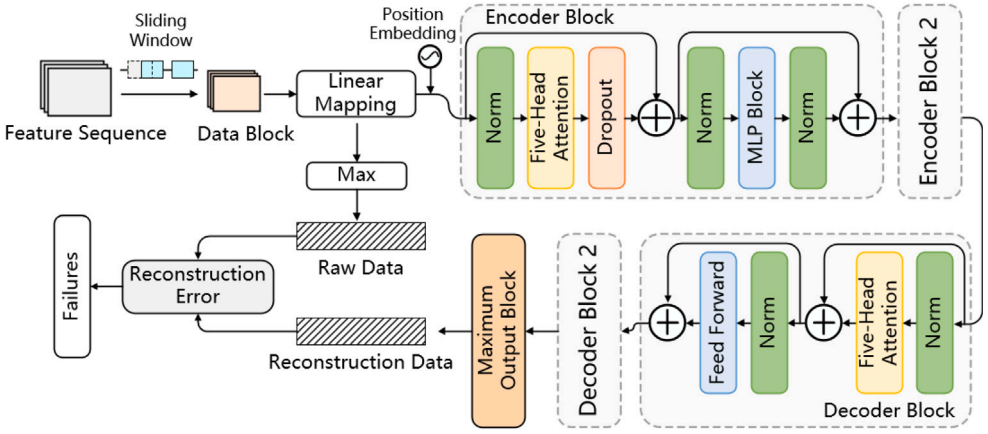


Fig. 7. Transformer-based failure analysis in MSoFSAnomaly.

and feedforward neural networks to generate the reconstruction matrix of raw input embedding z_d. In addition, since the F features can significantly increase in value when a fault occurs, we compute the maximum value of the F features corresponding to M microservices in $z_d \in \mathbb{R}^{W\times(M\cdot F)}$ as the reconstructed output. This approach helps us capture anomalies while reconstructing normal data. Eq. (5) shows the computation process for the output block.

$$\text{Output} = \text{Max}(z_d[:, i]) \quad i = 1, 2, \ldots, M \cdot F \tag{5}$$

4.4.3. Failure alarm

The failure alarm in MSoFSAnomaly needs to accurately detect a small number of continuous anomalies within a large amount of test data while minimizing false positives and negatives. To achieve this, we need to assess the probability of anomalies in each data block. The data D_w^t, with a window size at time t, is input into the encoder to obtain the embedding vector z_0^t. This vector is fed into the decoder and the output layer to generate the reconstructed matrix Rec_w^t. We extract the maximum value of each feature for every microservice from each window to characterize the raw data features as Max_w^t.

We determine whether a fault exists in a data block based on Max_w^t and Rec_w^t. Our work follows the assumption that if a fault exists, the distance between Max_w^t and Rec_w^t is significantly larger than the baseline from normal data. Based on this assumption, our approach uses the reconstruction error between Max_w^t and Rec_w^t to measure the anomaly probability. The loss function is shown in Eq. (6).

$$\text{Loss} = \frac{\sum_i^M \sum_j^F (\text{Rec}_w^t[i,j] - \text{Max}_w^t[i,j])^2}{M \cdot F} \tag{6}$$

The detection of anomalies relies on the threshold of normal data. Existing methods (Li et al., 2023b) use the distance between input data and reconstructed data as the anomaly threshold, which has been proven effective. We adopt this idea by using the maximum loss from the validation dataset at the end of the training, calculated in Eq. (6). This loss characterizes the distance between the input and reconstructed data and serves as the basis for our threshold, denoted as val_{re}.

Furthermore, we randomly select 2 h of test data from both systems and visualize the reconstructed outputs to analyze the distribution of reconstruction errors. Fig. 8 shows that the reconstruction outputs of normal data in the test set are clustered within a specific range. In contrast, MSoFSAnomaly demonstrates a different capacity for capturing anomalously enlarged features. Thus, the reconstructed outputs for anomalous data exhibit a wider range. Specifically, for TT data, there is a subset where the reconstruction of anomalous data is similar in size to that of the normal data. This result indicates that MSoFSAnomaly fails to capture the feature changes associated with this fault.

From the distribution of reconstruction outputs, we find that using a simple linear threshold yields satisfactory detection results. Our experiments reveal a difference in the distribution of reconstruction outputs for normal data in the training set compared to the test set. This discrepancy arises because anomalies may influence the normal data in the test set. If we use the reconstruction error from the validation set as a threshold, some normal data may be misclassified as anomalies. To address this, we introduce a parameter μ to set the threshold based on val_{re}, as shown in Eq. (7). The selection of μ will be discussed in Section 5.8.

$$test_{th} = val_{re} + \mu \cdot val_{re} \tag{7}$$

4.4.4. Failure interpretation

After a fault is detected, our approach uses an interpreter to analyze the microservices and specific features that significantly contribute to the failure detection.

At time t, the model processes the raw data, $D_w^t \in \mathbb{R}^{W\times(M\cdot F)}$, to produce the reconstructed data Rec_w^t. The maximum value of each feature extracted from the raw window data is Max_w^t. Along the time dimension, we apply a linear projection to Rec_w^t and Max_w^t to obtain the $R_w^t \in \mathbb{R}^{1\times(M\cdot F)}$ and $M_w^t \in \mathbb{R}^{1\times(M\cdot F)}$, respectively. We construct a distance matrix Dis by calculating the sum of the squared differences between corresponding elements in R_w^t and M_w^t. The Dis represent the degree of feature change for different microservices. The calculation process is in Eq. (8).

$$Dis_W^t[i] = \left(R_w^t[i] - M_w^t[i]\right)^2, \quad i = 1, 2, \ldots, M \cdot F \tag{8}$$

We use Dis to represent the contributions to failure detection. Each value in Dis corresponds to the contribution of a specific feature in a specific microservice to failure detection. A higher value indicates a more significant contribution. Therefore, following existing studies, we adopt a top-K ranking strategy with K = 3 to select the top three values, identifying the microservices and features associated with faults.

5. Evaluation

In this section, we conduct experiments to answer the following research questions:

- **RQ1:** How does our method perform in terms of different faults, features and data?
- **RQ2:** Can our method outperform baselines for failure analysis in microservice systems by reducing false positives while guaranteeing Recall?
- **RQ3:** How does our method perform in terms of efficiency and scalability?
- **RQ4:** How do the selected fault-sensitive microservices influence our method performance?
- **RQ5:** How do the hyperparameters influence our method performance?

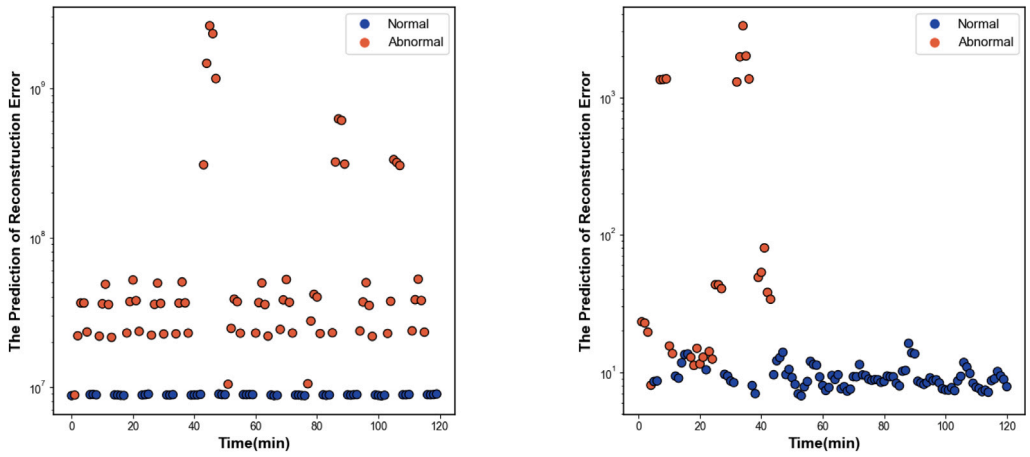


Fig. 8. The difference of reconstructed outputs between normal and anomalous data.

5.1. Experiment setup

5.1.1. Microservice systems

We deploy two open-source microservice benchmarks, TrainTicket (TT) (Zhou et al., 2018) and OnlineBoutique (OB) (Googlecloudplatform, 2021), on our experimental platform. TrainTicket (Zhou et al., 2018) is a ticket booking system with 41 microservices, including order, price, and seat services. OnlineBoutique (Googlecloudplatform, 2021) is an e-commerce microservice system with 11 microservices where users can browse items, add them to their carts, and make purchases. Both systems have been widely used in previous studies. In addition, we use a public dataset from a Social Network system (Gan et al., 2019), which is a broadcast-style social networking site with 21 microservices.

5.1.2. Experimental platform

We deploy the two microservice systems on a Kubernetes cluster with four cloud servers, each equipped with a 4-core 2.50 GHz CPU, 16 GB of memory, and running CentOS 7.6. We also deploy the open-source tool Jaeger (Jaeger, 2024) to collect traces, along with cAdvisor (Google, 2024) and Istio (Istio, 2024) to collect metrics such as CPU-related metrics and request latency.

5.1.3. Datasets

To simulate system or application issues in our deployed microservice systems, we use fault injection following previous works (Yu et al., 2023; Li et al., 2022, 2023a). For resource faults, such as CPU consumption, and network faults, such as network delay, we perform these two types of fault injection with Chaos Mesh (Chaos-Mesh, 2024), similar to existing works (Li et al., 2022, 2023a). Specifically, we use the Stress Test of Chaos Mesh for resource faults and the Request/Response Delay of Chaos Mesh for network delay. For program code faults, we utilize Chaos Mesh (Chaos-Mesh, 2024), Hypno (kmaork, 2024), and Failpoint (pingcap, 2019) to inject faults into services written in Java, Python, and Golang, following the existing works (Yu et al., 2023; Zhang et al., 2022).

We randomly select microservices and inject a fault into each selected microservice. For example, we randomly selected 12 different microservices to inject faults in the Train Ticket system. The selected services are preserve, security, contacts, verification-code, food, trainfood, travel, travel2, basic, price, route, and seat. In the OnlineBoutique system, we select 10 from 11 microservices, excluding the loadgenerator service, which uses Python/Locust to send requests that imitate user shopping flows to the frontend. We set each fault duration to 3 min following the existing work (Yu et al., 2023). The details of two datasets from Train Ticket (Zhou et al., 2018) and OnlineBoutique (Googlecloudplatform, 2021) are shown in Table 4. In addition, the SN Data is a public dataset published by Eadro (Lee et al., 2023) from a microservice benchmark Social Network (Gan et al., 2019). We use metrics from SN Data to demonstrate the performance of current failure detectors in the Introduction section. The information is also shown in Table 4.

We used three data sets from the online store (OB), train ticket (TT), and social network (SN) systems to evaluate failure detection performance. Table 4 provides details of these datasets.

5.1.4. Implementation

We implement MSoFSAnomaly using Python 3.10. All experiments are conducted on a Linux server with a 3.20 GHz CPU, 132 GB of total memory, running Ubuntu 20.04. We divide the dataset into training, validation, and testing in a 7:1:2 ratio to evaluate the performance of MSoFSAnomaly. For the hyperparameters of MSoFSAnomaly, we use 5-head self-attention and train the model using the Adam optimizer with an initial learning rate of 0.001. We adjust the learning rate with StepLR, use a batch size 32, and set the epoch number to 100. Besides, the window size is 3, as discussed in Section 5.8.

5.2. Methods to compare

We use the following eight unsupervised metric-based and trace-based failure detection approaches as the baselines.

- **N-Sigma** (Lin et al., 2018) is a metric-based failure detection approach by computing the mean (ρ) and the standard deviation (σ) of historical fault-free data.
- **SPOT** (Siffer et al., 2017) is a univariate-based algorithm for time-series anomaly based on the Extreme Value Theory (EVT).
- **JumpStarter** (Ma et al., 2021) is a metric-based approach and applies a compressed sensing technique for failure detection.
- **ADSketch** (Chen et al., 2022) is a metric-based approach that involves normal pattern mining and pattern comparison.
- **ServiceAnomaly** (Panahandeh et al., 2024) combines traces and six metrics to build call propagation graphs and uses the graph matching method to detect anomalous calls or metrics.
- **MicroHECL-SVM** (Liu et al., 2021) uses the one-class support vector machine (OC-SVM) to train a model for anomaly by analyzing response time fluctuations.
- **TraceAnomaly** (Liu et al., 2020) provides a trace-based approach to learn normal trace patterns and identify anomalies. Initially, normal and anomalous data are input into the model independently.

Table 4
The detailed information of datasets. #Micro denotes the number of microservices.

Data name	#Micro	Collection interval	Each fault duration	Fault injection	Anomaly ratio
OB data	11	1 min	3 min	56	31.11%
TT data	41	1 min	3 min	45	23.44%
SN data	21	1 s	2 min	72	21.88%

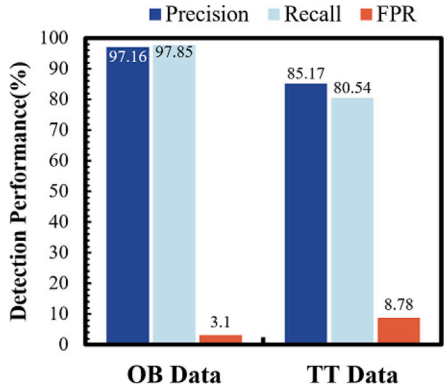


Fig. 9. Detection performance of MSoFSAnomaly.

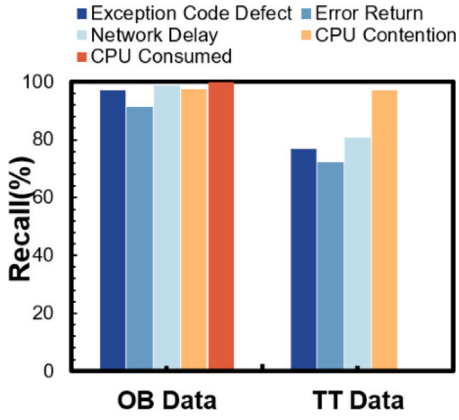


Fig. 10. Recall of MSoFSAnomaly for five faults.

- **LSTM-AD** (Nedelkoski et al., 2019) uses a single-modality and multimodal neural network with LSTM to learn the sequential nature of trace data. In our experiment, we apply the single-modality LSTM.

5.3. Evaluation metrics

To evaluate the quality of data fed into localizers after failure detection, we use Precision, False Positive Rate (FPR), and True Positive Rate (TPR), also called Recall, on the test datasets.

5.4. Detection capabilities of MSoFSAnomaly (RQ1)

Fig. 9 shows our method's overall failure detection capabilities on OB and TT data. For OB data, MSoFSAnomaly achieves a Precision of 97.16%, a Recall of 97.85%, and an FPR of 3.10%. For TT data, the Precision is 85.17%, the Recall is 80.54%, and the FPR is 8.78%. Overall, the performance of MSoFSAnomaly is significantly better in the OB system compared to the TT system.

We conduct an in-depth analysis of the raw feature distributions and discover that the normal data utilized for training in the OB system is more densely clustered than that in the TT system. This discovery implies that MSoFSAnomaly has greater ease in identifying the precise boundaries of normal data for the OB data. Additionally, as depicted in the visualization of the reconstructed normal and anomalous data in Section 4.4.3, we observe that some normal and anomalous data in the TT dataset are incredibly close. The linear threshold employed by MSoFSAnomaly might result in the misclassification of these data points. These two factors contribute to the reduced efficacy of MSoFSAnomaly in the TT data.

Fig. 10 details the MSoFSAnomaly's detection capabilities for five faults. In the OB data, MSoFSAnomaly achieves high recall scores of 97.14%, 91.43%, 98.75%, 97.5%, and 100% for the exception code defect, error return, network delay, CPU contention, and CPU consumed faults. In contrast, in the TT data, MSoFSAnomaly attains recall scores of 76.69%, 72.22%, 80.88%, and 97.14% for the first four faults.

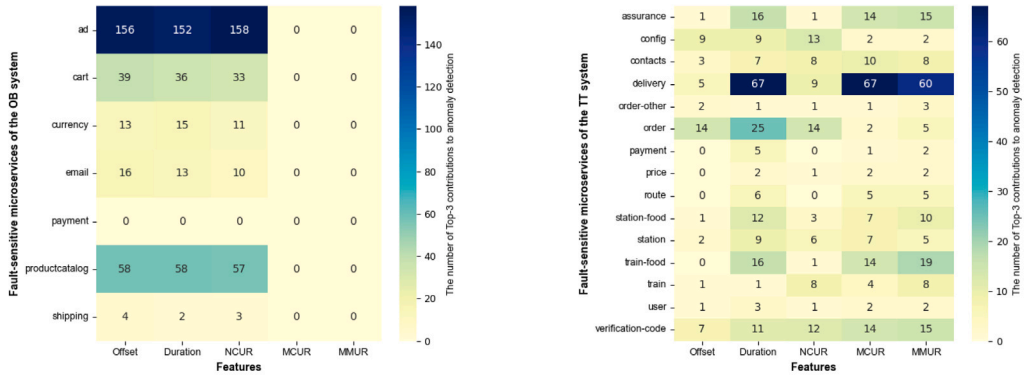
MSoFSAnomaly achieves the highest detection performance for CPU contention, and CPU consumed faults because these two faults significantly increase the MCUR and MMUR features. This result indicates a strong correlation between these faults and the two features. In addition, network delay faults notably increase traces' Span Duration and Span Offset features. Compared to the above three faults, the exception code defect and error return faults are more challenging to detect using metrics. We find that combining the two trace-related features enhances failure detection for these two application-level faults.

Based on the detection results, MSoFSAnomaly uses a failure interpreter to measure the contribution of fault-sensitive microservices and their features to the detection outcomes. Specifically, MSoFSAnomaly first uses Eq. (8) to calculate the distance Dis between the raw and reconstructed data. Each distance value represents the change in each feature of a microservice compared to the raw feature. A larger change indicates a higher probability of an anomaly. Then, for each correctly detected fault, MSoFSAnomaly extracts the top three values in Dis, with each value representing the change in features corresponding to a specific microservice. We count the number of (microservice, feature) tuples to create a heat map, as shown in Fig. 11. In other words, if a change of a feature F_i of a microservice M_i appears in the extracted top three records for a fault, we increment the corresponding element for the (M_i, F_i) pair by 1. In this figure, darker colors indicate a higher contribution from a specific microservice's features.

For the OB data, we find three features—Span Offset, Span Duration, and NCUR—have the highest contributions. Among the seven fault-sensitive microservices, "ad", "productcatalog", and "cart" are more prone to faults. In addition, different features contribute differently to various faults. For example, we find that NCUR has the greatest contribution for resource faults, while Span Offset and Duration are most significant for network delay and code faults. For the TT data, the microservices "delivery" and "train-food" correlate with Span Duration, MCUR, and MMUR features. At the same time, the microservice "order" is linked to Span Duration, Span Offset, and NCUR. We find that for resource faults, MCUR and MMUR have the greatest contributions, while for network delay and code faults, Span Duration is the most significant.

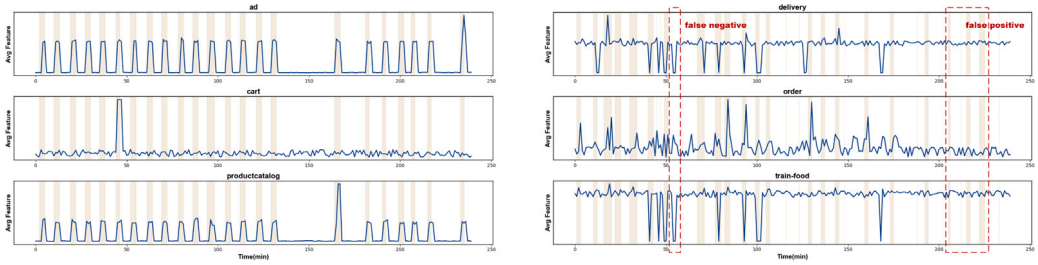
MSoFSAnomaly highlights these cases to demonstrate that our approach can help system administrators focus their monitoring efforts on specific microservices and their features, providing crucial information to enhance system security. For example, in the OB system, administrators can focus on the Span Offset, Duration, and NCUR of the services "ad", "productcatalog", and "cart" to detect failure.

Existing studies lack the analysis of data distribution. In real network scenarios, normal and anomalous data are typically distributed in chronological order, with the amount of anomalous data being much



(a) OB data (b) TT data

Fig. 11. The contribution of fault-sensitive microservices and their features to the failure detection.



(a) OB data (b) TT data

Fig. 12. The consistency of feature changes (blue curves) and detection results (yellow box) for locally anomalous data.

smaller than that of normal data. In this paper, we refer to this type of data distribution as locally anomalous data and focus on it as our primary object.

To demonstrate MSoFSAnomaly's ability to detect locally anomalous data, we visualize its failure detection results. Since the raw data consists of a three-dimensional feature $D_{(T,M,F)}$, we compute the average values for the five features associated with each microservice to construct a sequence of averages over time for visualization on a flat surface. Based on the results in Fig. 12, we select "ad", "productcatalog", and "cart" in OB data and "delivery", "order", and "train-food" in TT data. To effectively visualize the detection performance of MSoFSAnomaly, we randomly select a continuous 4-h window during the fault occurrence stage to visualize detection capabilities based on the evaluation using the entire test dataset. Fig. 12 shows the relationships between detection results and feature changes from the emergence of faults to their fix. The yellow boxes in Fig. 12 indicate anomalies detected by MSoFSAnomaly, and the blue curves represent the average values of features.

Fig. 12 illustrates how MSoFSAnomal adapts to locally anomalous data. For the OB data, the results align well with changes in feature representations. At any given moment, if any feature of the targeted microservice exhibits an anomalous behavior, it indicates that the system is experiencing failures.

In contrast, for the TT data, while some detection results are consistent with feature changes, there are still false positives and negatives. As shown in Fig. 9, the false negative rate is around 19.46%. This result occurs because some anomalies induce feature changes that do not significantly differ from normal data features. Additionally, the false positive rate is about 8.78%.

Our in-depth analysis of misclassified samples reveals that data labeled as normal for a certain period following anomalous events may still exhibit anomalous feature distributions due to the lingering effects of those anomalies. Specifically, such data appears shortly after a fault occurs, as the system undergoes recovery or gradual resource state transitions, retaining some anomalous characteristics.

For instance, TT data is collected every minute, with each fault injected for three minutes, leading us to label the data collected during this period (A, B, C) as anomalous, while data afterward is labeled as normal. However, further analysis reveals that adjacent data following this period (e.g., D, E, F) may still exhibit anomalous characteristics. We refer to this type of test data—labeled as normal but potentially influenced by anomalies—as "pseudo-normal" data.

During testing, our method detects these pseudo-normal data—labeled as normal but exhibiting anomalous characteristics—as anomalies. This does not indicate a misclassification but rather accurately reflects the impact of system faults on subsequent states. In real-world scenarios, the anomaly in such data is inherently present, and correctly identifying these states helps provide more accurate anomaly data for root cause analysis.

In summary, our method achieves high Precision and Recall, along with a low FPR, for different fault types, demonstrating its effectiveness in detecting anomalies in microservice systems.

5.5. Overall performance (RQ2)

Table 5 lists the average Precision, Recall, and FPR for MSoFSAnomaly and eight baseline approaches on two datasets. MSoFSAnomaly outperforms all baseline methods, achieving an optimal balance between Recall, Precision, and FPR. For the TT data, although our approach decreases Recall, it significantly improves Precision and reduces FPR, demonstrating that our method is reliable and effective for practical applications.

Table 5
The average Precision, FPR, and Recall of different approaches on two datasets (viz., ↑ stands for higher values represent better performance and ↓ is the opposite).

Approach	Technique	Source		OB data			TT data		
		Metric	Trace	Precision ↑	FPR ↓	Recall ↑	Precision ↑	FPR↓	Recall ↑
N-Sigma (Lin et al., 2018)	Statistical-based	✓		34.10%	31.58%	34.71%	23.73%	88.21%	89.63%
SPOT (Siffer et al., 2017)	EVT-based	✓		33.80%	39.66%	42.35%	21.87%	51.02%	46.67%
JumpStarter (Ma et al., 2021)	Compressed-sensing technique	✓		40.54%	34.92%	46.88%	33.83%	87.13%	88.24%
ServiceAnomaly (Panahandeh et al., 2024)	Graph matching	✓	✓	61.22%	33.93%	53.57%	53.25%	80.00%	91.11%
ADSketch (Chen et al., 2022)	Pattern matching	✓	✓	82.81%	10.65%	80.95%	42.68%	35.05%	73.02%
MicroHECL-SVM (Liu et al., 2021)	ML-based	✓	✓	45.61%	49.87%	92.86%	35.11%	49.89%	88.15%
TraceAnomaly (Liu et al., 2020)			✓	79.93%	24.81%	91.07%	80.70%	12.45%	83.19%
LSTM-AD (Nedelkoski et al., 2019)	DL-based		✓	81.44%	9.68%	94.05%	63.83%	15.42%	88.89%
Ours		✓	✓	**97.16%**	**3.10%**	**97.85%**	**85.17%**	**8.78%**	**80.54%**

Table 6
The performance of MSoFSAnomaly on different source data (viz., ↑ stands for higher values represent better performance and ↓ is the opposite).

Data source	OB data			TT data		
	Precision ↑	FPR ↓	Recall ↑	Precision ↑	FPR↓	Recall ↑
Ours w/o traces	50.50%	76.37%	71.79%	37.67%	65.16%	62.90%
Ours w/o metrics	85.32%	18.60%	95.71%	57.64%	34.56%	75.11%
Ours	**97.16%**	**3.10%**	**97.85%**	**85.17%**	**8.78%**	**80.54%**

5.5.1. Multi-source data for failure detection

Our approach achieves the best detection results when using metric and trace data. In contrast, Table 6 shows the performance with metrics alone is significantly lower than when using only trace or multi-source data.

Besides, by comparing the data and detection performance of other baseline methods, we discover that leveraging multi-source data to reflect faults can significantly improve overall failure detection performance. These findings are consistent with those described in Section 4.3. Features only from metrics cannot reflect network and program code faults, making the metric-only approach ineffective for accurately identifying both types of faults.

5.5.2. System-based failure detection

N-Sigma (Lin et al., 2018), SPOT (Siffer et al., 2017), and JumpStarter (Ma et al., 2021) are instance-based failure detection methods. In our experiments, we construct a detection model for each feature of every fault-sensitive microservice and analyze the results at each time point. However, the detection performance of these three approaches on the two datasets is unsatisfactory. They treat each microservice as independent, ignoring the interactions between them. Additionally, all three methods rely solely on metric data for failure detection, making them unable to reflect network and program code faults.

In contrast, methods that utilize trace or multi-source data are system-based. These approaches analyze multiple microservice features, such as call relations and latencies, to identify anomalies from a system-wide perspective. System-based methods outperform instance-based approaches by capturing the relations between microservices, enabling more accurate and comprehensive failure detection. Furthermore, the system-based approach requires only one model for the entire system rather than separate models for each microservice feature, significantly reducing the time needed to construct and update normal baselines.

5.5.3. ML-based failure detection

ML-based methods perform better than the first five methods listed in Table 5. The methods SPOT (Siffer et al., 2017), and N-Sigma (Lin et al., 2018) detect anomalies based on the statistical distribution characteristics of each feature in each microservice. They learn limited patterns and use simple thresholds for anomaly identification. This limit makes these two methods very sensitive to anomalies, often resulting in many normal samples being misclassified as anomalies. Consequently, they show unsatisfactory levels of Precision and FPR. In contrast, JumpStarter (Ma et al., 2021), ServiceAnomaly (Panahandeh et al., 2024), and ADSketch (Chen et al., 2022) employ techniques like compressed sensing, graph matching, and pattern matching to detect anomalies. These methods can capture more normal patterns compared to the statistical methods, improving overall detection performance.

MicroHECL-SVM (Liu et al., 2021) is a ML-based approach that achieves failure detection by constructing linear bounds on normal data. However, it inevitably produces false positives while maintaining detection performance on anomalous data. This issue arises because OC-SVM relies on significant differences between normal and anomalous data features. The boundaries created from the normal data during training may not accurately capture the distinctions between normal and anomalous data with similar features in the test set.

TraceAnomaly (Liu et al., 2020), LSTM-AD (Nedelkoski et al., 2019), and our method are based on deep learning, which can adapt to complex data distribution by learning nonlinear feature boundaries. In our experiments, we discover that TraceAnomaly (Liu et al., 2020) adapts to datasets with a balanced distribution of normal and anomalous data. It mainly focuses on the response time series of all spans in a trace at a time point, ignoring feature changes between neighboring time points. Thus, TraceAnomaly (Liu et al., 2020) performs lower than the other two deep learning methods in the locally anomalous dataset used in this paper.

Our approach, MSoFSAnomaly, focuses on contextual feature changes in the time dimension and uses an attention mechanism to highlight anomalous increases in different features related to various fault-sensitive microservices.

In summary, our method outperforms baseline approaches for failure analysis in microservice systems by reducing false positives while ensuring high Recall. This effectively balances Precision and Recall, enhancing overall failure detection performance.

5.6. Efficiency and scalability (RQ3)

Based on the detection results in Table 5, we compare the four ML-based approaches. Model training can be done offline, but in real online scenarios, the focus is on the model's testing efficiency. Therefore, we compare the testing time of the four methods using the same size of test data, with the results shown in Table 7.

MicroHECL-SVM (Liu et al., 2021), as an ML-based method, primarily calculates the distance between support vectors and input samples during testing, which involves only basic linear operations. Our method is the fastest among the three deep learning approaches, taking approximately 0.03 s on both datasets. In contrast, TraceAnomaly (Liu

Table 7
Testing time costs for four ML-based methods.

Approach	OB data(s)	TT data(s)
MicroHECL-SVM (Liu et al., 2021)	0.007	0.007
TraceAnomaly (Liu et al., 2020)	0.516	0.8912
LSTM-AD (Nedelkoski et al., 2019)	0.499	0.884
Ours	**0.030**	**0.031**

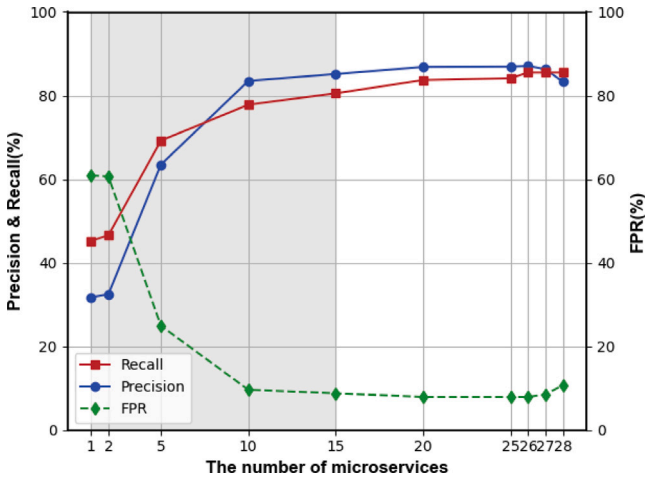


Fig. 13. Performance with different microservice scales.

et al., 2020), due to the computational complexity of Variational Auto-Encoders (VAE) combined with a modified version of Glow's flow, takes about 0.5 to 0.9 s. Meanwhile, LSTM-AD (Nedelkoski et al., 2019) processes sequences step by step, resulting in testing times of 0.499 s on the OB data and 0.884 s on the TT data.

Furthermore, Fig. 13 shows that MSoFSAnomaly's detection performance significantly improves as the number of microservices increases from 1 to 15. When the number of microservices exceeds 15, the detection performance remains relatively stable. This stability suggests that our approach can effectively manage more microservices without a significant drop in detection performance.

Overall, our method has low testing time and good scalability, adapting well to an increasing number of microservices.

5.7. Performance of fault-sensitive microservices (RQ4)

The microservices represented in the gray parts in Fig. 13 indicate the selected fault-sensitive microservices. When we input the feature sequences from these 15 selected microservices in the TT system into MSoFSAnomaly, the detection performance is comparable to inputting feature sequences from all microservices. This result suggests that our selection method can achieve satisfactory detection results with minimal resource cost.

Besides, we compare the contributions of fault-sensitive microservices versus other microservices to MSoFSAnomaly's performance. Fig. 14 indicates that the selected fault-sensitive microservices are more effective at reflecting faults and contribute more to failure detection.

In summary, using selected fault-sensitive microservices for failure detection significantly outperforms others. Our approach is well-suited for real microservice systems, enabling satisfactory failure detection performance by analyzing multi-source data from fault-sensitive microservices without requiring data from all microservices.

5.8. Hyperparameters sensitivity analysis (RQ5)

We mainly discuss the effect of two hyperparameters in the threshold set and data block construction on MSoFSAnomaly's performance

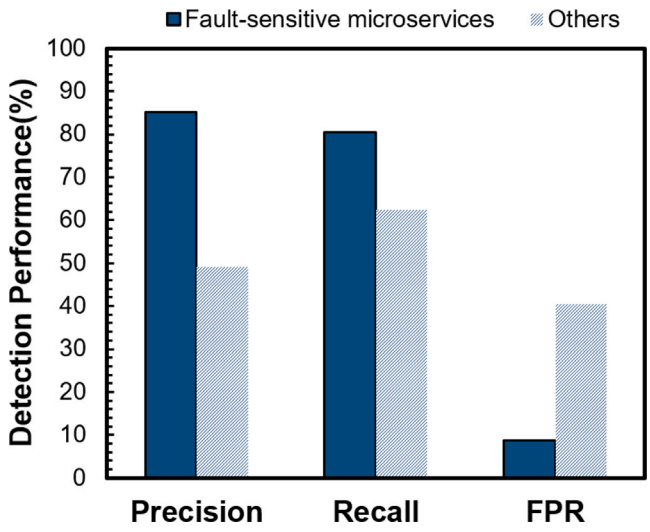


Fig. 14. Performance with different microservice sets.

in the OB data. Fig. 15 shows how the Precision, Recall, and FPR of MSoFSAnomaly change with different values of hyperparameters. Specifically, we increase the size of the constant coefficient in threshold set μ from 1 to 13. From the experiment results, we can find that if μ is too large, the high threshold makes it easy for anomalous data to be misclassified as normal; if μ is too small, the low threshold makes normal data easily misclassified as anomalous data. μ between 10 and 12 can lead to relatively good performance. Thus, we set $\mu = 11$ in OB data.

We change the size of window W in data block construction before feature embedding. Since the collection interval of OB data is 1 min and the duration of each fault is 3 min, we set the window size from 1 to 4. The W of 3 yields the best performance. If W is too small, the model may not be able to capture the complete fault information; if W is too large, a data block may contain too much irrelevant data, resulting in difficulty for the model to focus on fault-related features, with important local anomalous changes may be diluted.

Through sensitivity analysis of the hyperparameters, we finally set the threshold hyperparameter μ to 11 and the window size W to 3 in our paper.

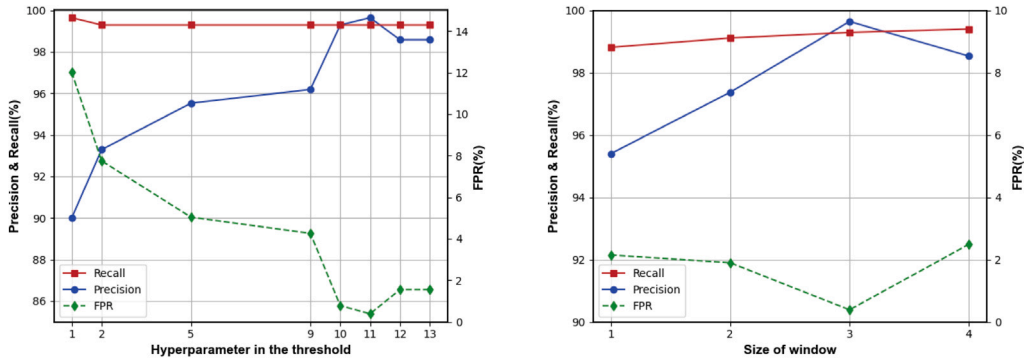
6. Discussion

6.1. Challenges and future work

One challenge in microservice anomaly detection is the variability in architectural designs, which may influence the choice of practical features for failure detection. While our study employs widely used features—such as resource utilization and trace duration—different microservice architectures (e.g., service mesh, serverless, and edge computing) may exhibit unique behaviors that require architecture-specific feature selection. Future research should explore adaptive feature selection techniques to optimize detection across diverse architectures.

Another challenge is the generality of anomaly detection mechanisms. As real-word microservice systems expand, maintaining efficiency in service selection and anomaly detection becomes critical. Optimizing computational efficiency, especially in large-scale deployments, remains an open area for improvement.

Last, MSoFSAnomaly selects fault-sensitive microservices for failure detection, primarily based on the analysis of four fault propagation patterns and experimental validation. However, it does not provide a theoretical reasoning for this microservice selection. Future work can construct a mathematical model from a probabilistic perspective to validate the effectiveness of microservice selection.



(a) μ (b) W

Fig. 15. Performance under different parameters.

6.2. Threats to validity

The main Internal threat comes from the normal data used for training. When this normal data contains noise or lacks certain types of faults, the detection performance of MSoFSAnomaly may be affected. To address this issue, existing methods (Yu et al., 2023) propose replaying and capturing system workloads to construct normal data. In addition, even without replaying workloads, collecting normal data is relatively easy because the production systems are mainly in normal status.

The threat to the conclusion validity primarily uses the validation dataset's maximum reconstruction loss to establish the failure detection threshold. The selection of the parameter μ in this threshold can affect the detection performance of MSoFSAnomaly. Based on our hyperparameters sensitivity analysis, we set μ to 11.

The external threat mainly comes from the experimental environment. We evaluate our method on two widely-used microservice systems containing 11 and 41 microservices, respectively. It remains unclear whether the performance of MSoFSAnomaly can be generalized to more complex real-world systems. To alleviate the complexity of fault types in real systems, we follow existing methods (Yu et al., 2023) and inject resources, network issues, and program code defects from industrial systems into the microservices (Li et al., 2022). However, further efforts are needed to validate the effectiveness of MSoFSAnomaly in complex systems with more microservices.

7. Conclusion

Failure analysis in microservice systems is crucial for maintaining service reliability. In this work, we propose MSoFSAnomaly, a failure analysis framework that leverages multi-source data from fault-sensitive microservices to detect failures.

Our method establishes correlations between faults and multi-source features, analyzes fault propagation patterns to identify fault-sensitive microservices, and compares their feature representations, enabling failure detection from a system perspective without requiring baselines for each microservice. MSoFSAnomaly uses a Transformer-based model to effectively identify locally anomalous data and provide pure anomalous data for localizers with a low false positive rate. By integrating a fault-sensitive design with multi-source data fusion and Transformer-based modeling, MSoFSAnomaly captures comprehensive and important features while limiting its attention to only the most relevant microservices.

Our results show that MSoFSAnomaly achieves superior performance with a well-balanced trade-off between precision and recall within acceptable time overhead. Moreover, it quantifies the contributions of individual microservices and key features to the detection process.

CRediT authorship contribution statement

Nan Fu: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Guang Cheng:** Supervision, Project administration, Funding acquisition. **Guangye Dai:** Writing – review & editing. **Hantao Mei:** Writing – original draft, Supervision. **Xing Qiu:** Writing – original draft, Supervision. **Yue Teng:** Supervision.

Declaration of competing interest

We declare that we have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the General Program of the National Natural Science Foundation of China under grant number 62172093.

Data availability

Data will be made available on request.

References

Audibert, J., Michiardi, P., Guyard, F., Marti, S., Zuluaga, M.A., 2020. Usad: Unsupervised anomaly detection on multivariate time series. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 3395–3404.

Chaos-Mesh, 2024. Chaos-mesh: A chaos engineering platform for kubernetes. Github, https://github.com/chaos-mesh/chaos-mesh. (Accessed 15 April 2024).

Chen, Z., Kang, Y., Li, L., Zhang, X., Zhang, H., Xu, H., Zhou, Y., Yang, L., Sun, J., Xu, Z., et al., 2020. Towards intelligent incident management: why we need it and how we make it. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 1487–1497.

Chen, Z., Liu, J., Su, Y., Zhang, H., Ling, X., Yang, Y., Lyu, M.R., 2022. Adaptive performance anomaly detection for online service systems via pattern sketching. In: Proceedings of the 44th International Conference on Software Engineering. pp. 61–72.

FudanSELab, 2022. Train ticket - a benchmark microservice system. GitHub, https://github.com/FudanSELab/train-ticket. (Accessed 15 April 2024).

Gan, Y., Zhang, Y., Cheng, D., Shetty, A., Rathi, P., Katarki, N., Bruno, A., Hu, J., Ritchken, B., Jackson, B., et al., 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 3–18.

Google, 2024. Cadvisor. GitHub, https://github.com/google/cadvisor. (Accessed 16 April 2024).

2021. GoogleCloudPlatform/microservices-demo. URL https://github.com/GoogleCloudPlatform/microservices-demo. (Accessed 15 April 2025).

Hao, X., Chen, Y., Yang, C., Du, Z., Ma, C., Wu, C., Meng, X., 2024. From chaos to clarity: Time series anomaly detection in astronomical observations. arXiv preprint arXiv:2403.10220.

Istio, 2024. The istio service mesh. Istio, https://istio.io/. (Accessed 15 April 2024).

Jaeger, 2024. Jaeger: open source, end-to-end distributed tracing. Jaegertracing.io, https://www.jaegertracing.io. (Accessed 16 April 2024).

Kim, M., Sumbaly, R., Shah, S., 2013. Root cause detection in a service-oriented architecture. ACM Sigmetrics Perform. Eval. Rev. 41 (1), 93–104.

kmaork, 2024. GitHub - kmaork/hypno: Inject python code into a running python process. URL https://github.com/kmaork/hypno?tab=readme-ov-file (Accessed 15 April 2025).

Kraskov, A., Stögbauer, H., Grassberger, P., 2004. Estimating mutual information. Phys. Rev. E— Stat. Nonlinear Soft Matter Phys. 69 (6), 066138.

Lee, C., Yang, T., Chen, Z., Su, Y., Lyu, M.R., 2023. Eadro: An end-to-end troubleshooting framework for microservices on multi-source data. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE, pp. 1750–1762.

Li, Z., Chen, J., Chen, Y., Luo, C., Zhao, Y., Sun, Y., Sui, K., Wang, X., Liu, D., Jin, X., et al., 2023a. Generic and robust root cause localization for multi-dimensional data in online service systems. J. Syst. Softw. 203, 111748.

Li, Z., Chen, J., Jiao, R., Zhao, N., Wang, Z., Zhang, S., Wu, Y., Jiang, L., Yan, L., Wang, Z., et al., 2021. Practical root cause localization for microservice systems via trace analysis. In: 2021 IEEE/ACM 29th International Symposium on Quality of Service. IWQOS, IEEE, pp. 1–10.

Li, Y., Lu, Y., Wang, J., Qi, Q., Wang, J., Wang, Y., Liao, J., 2023b. Tadl: Fault localization with transformer-based anomaly detection for dynamic microservice systems. In: 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, pp. 718–722.

Li, Z., Zhao, N., Li, M., Lu, X., Wang, L., Chang, D., Nie, X., Cao, L., Zhang, W., Sui, K., et al., 2022. Actionable and interpretable fault localization for recurring failures in online service systems. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 996–1008.

Lin, J., Chen, P., Zheng, Z., 2018. Microscope: Pinpoint performance issues with causal graphs in micro-service environments. In: Service-Oriented Computing: 16th International Conference, ICSOC 2018, Hangzhou, China, November 12-15, 2018, Proceedings 16. Springer, pp. 3–20.

Liu, D., He, C., Peng, X., Lin, F., Zhang, C., Gong, S., Li, Z., Ou, J., Wu, Z., 2021. Microhecl: High-efficient root cause localization in large-scale microservice systems. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, pp. 338–347.

Liu, H., Huang, X., Jia, M., Jia, T., Han, J., Li, Y., Wu, Z., 2024. UAC-AD: Unsupervised adversarial contrastive learning for anomaly detection on multi-modal data in microservice systems. IEEE Trans. Serv. Comput..

Liu, P., Xu, H., Ouyang, Q., Jiao, R., Chen, Z., Zhang, S., Yang, J., Mo, L., Zeng, J., Xue, W., et al., 2020. Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks. In: 2020 IEEE 31st International Symposium on Software Reliability Engineering. ISSRE, IEEE, pp. 48–58.

Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., Ding, Y., He, J., Xu, C., 2021. Characterizing microservice dependency and performance: Alibaba trace analysis. In: Proceedings of the ACM Symposium on Cloud Computing. pp. 412–426.

Ma, M., Zhang, S., Chen, J., Xu, J., Li, H., Lin, Y., Nie, X., Zhou, B., Wang, Y., Pei, D., 2021. {Jump-starting} multivariate time series anomaly detection for online service systems. In: 2021 USENIX Annual Technical Conference (USENIX ATC 21). pp. 413–426.

Meng, Y., Zhang, S., Sun, Y., Zhang, R., Hu, Z., Zhang, Y., Jia, C., Wang, Z., Pei, D., 2020. Localizing failure root causes in a microservice through causality inference. In: 2020 IEEE/ACM 28th International Symposium on Quality of Service. IWQoS, IEEE, pp. 1–10.

Nedelkoski, S., Cardoso, J., Kao, O., 2019. Anomaly detection from system tracing data using multimodal deep learning. In: 2019 IEEE 12th International Conference on Cloud Computing. CLOUD, IEEE, pp. 179–186.

Notaro, P., Cardoso, J., Gerndt, M., 2021. A survey of aiops methods for failure management. ACM Trans. Intell. Syst. Technol. (TIST) 12 (6), 1–45.

Pan, Y., Ma, M., Jiang, X., Wang, P., 2023. DyCause: Crowdsourcing to diagnose microservice kernel failure. IEEE Trans. Dependable Secur. Comput. 20 (6), 4763–4777.

Panahandeh, M., Hamou-Lhadj, A., Hamdaqa, M., Miller, J., 2024. ServiceAnomaly: An anomaly detection approach in microservices using distributed traces and profiling metrics. J. Syst. Softw. 209, 111917.

pingcap, 2019. GitHub - pingcap/failpoint: An implementation of failpoints for golang. URL https://github.com/pingcap/failpoint (Accessed 15 April 2025).

Ren, R., Wang, Y., Liu, F., Li, Z., Tyson, G., Miao, T., Xie, G., 2023. Grace: Interpretable root cause analysis by graph convolutional network for microservices. In: 2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS). IEEE, pp. 1–4.

2024. Root cause analysis explained: Definition, examples, and methods. URL https://www.tableau.com/analytics/what-is-root-cause-analysis#definition. (Accessed 15 April 2025).

Siffer, A., Fouque, P.-A., Termier, A., Largouet, C., 2017. Anomaly detection in streams with extreme value theory. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1067–1075.

Su, Y., Zhao, Y., Niu, C., Liu, R., Sun, W., Pei, D., 2019. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 2828–2837.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30.

Wang, P., Xu, J., Ma, M., Lin, W., Pan, D., Wang, Y., Chen, P., 2018. Cloudranger: Root cause identification for cloud native systems. In: 2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing. CCGRID, IEEE, pp. 492–502.

Weng, J., Wang, J.H., Yang, J., Yang, Y., 2018. Root cause analysis of anomalies of multitier services in public clouds. IEEE/ACM Trans. Netw. 26 (4), 1646–1659.

Xu, J., Wang, Y., Chen, P., Wang, P., 2017. Lightweight and adaptive service api performance monitoring in highly dynamic cloud environment. In: 2017 IEEE International Conference on Services Computing. SCC, IEEE, pp. 35–43.

Yu, G., Chen, P., Chen, H., Guan, Z., Huang, Z., Jing, L., Weng, T., Sun, X., Li, X., 2021. Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments. In: Proceedings of the Web Conference 2021. pp. 3087–3098.

Yu, G., Chen, P., Li, Y., Chen, H., Li, X., Zheng, Z., 2023. Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 553–565.

Zhang, C., Peng, X., Sha, C., Zhang, K., Fu, Z., Wu, X., Lin, Q., Zhang, D., 2022. Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning. In: Proceedings of the 44th International Conference on Software Engineering. pp. 623–634.

Zhao, C., Ma, M., Zhong, Z., Zhang, S., Tan, Z., Xiong, X., Yu, L., Feng, J., Sun, Y., Zhang, Y., et al., 2023. Robust multimodal failure detection for microservice systems. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 5639–5649.

Zhou, X., Peng, X., Xie, T., Sun, J., Xu, C., Ji, C., Zhao, W., 2018. Benchmarking microservice systems for software engineering research. In: Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings. pp. 323–324.